

Stockbot / Capital Ledger — Architecture

A single-user automated equity trading system for acting on SPTulsian advisory calls.

Version as of 2026-09-02 · Ubuntu 22.04 on OCI · Python 3.10 · Oracle Autonomous Database

1. What this system is

SPTulsian is a paid Indian equity advisory. It emails stock recommendations on market mornings and publishes targets on a member portal. Acting on those calls by hand means reading email at 9am, deciding position size against a budget policy, placing buy orders, then remembering to place a sell trigger at the advisory's target — for every open position, every day.

This system does that work unattended, and keeps a ledger of what it did.

It is deliberately **not** a trading strategy. It does not decide *what* to buy; SPTulsian does. The system decides *how much*, subject to a written budget policy, executes reliably, and reports honestly on the outcome. A separate conviction layer assesses how well each recommendation is supported by public evidence, and since 2026-08-26 that layer **does** set position size and can refuse a buy outright (§2.2). It remains unvalidated — no backtest has yet shown it predicts returns — so its influence is deliberately confined to *how much*, never *what*.

Design priorities, in order

1. **Never place a wrong trade.** Ambiguity halts and asks a human.
2. **Never lose the ledger.** What happened must be recoverable and truthful.
3. **Fail loudly.** Silence must not be mistakable for a quiet day.
4. **Then, automate.**

Priority 1 is why the buy flow refuses to guess a ticker symbol. Priority 3 is why a scraper outage raises an alarm rather than simply producing no targets.

2. Functional view

2.1 The daily cycle

09:30 IST	RECOMMEND	Read overnight advisory email. Resolve each tip to a Kite trading symbol. Scrape the portal for target / timeframe. Write PENDING_BUY (clean) or NEEDS_REVIEW (ambiguous). No money moves. 90-minute review window
10:15 IST	CONVICTION	Score each new recommendation on public evidence. Sets the ticket size, and can refuse a buy. A failure here therefore HOLDS buying – which is why the watchdog checks for unscored trades.
10:45 IST	WATCHDOG 1	Are the buy INPUTS ready? Scraper alive, trades scored and attributed. Email if not.
11:00 IST	BUY	For each PENDING_BUY: price it, size it against the budget policy, place a real order. Re-attempt any NEEDS_REVIEW fixed in the window.
11:15 IST	WATCHDOG 2	Did the buy actually HAPPEN? Anything still PENDING_BUY means the run never reached it – a job that dies mid-flight still logs "complete".
16:00 IST	GTT	For filled buys with a target: place a GTT sell. For triggered GTTs: confirm the sell actually filled, then close the trade and recycle its budget.

The gap between 09:30 and 11:00 is the most important design decision in the system. Symbol resolution is where a catastrophic error lives — buying ICDSLTD (a ₹52 crore shell) when the advisory said CDSL (a ₹28,000 crore depository) is a real incident this system has had. Splitting recommendation from purchase creates a window in which a human can correct a bad symbol *before* money moves, rather than discovering it afterwards.

2.2 Position sizing

Each buy is capped by two independent rules, both read from the database:

Rule	Meaning
Category allocation	Each advisory section (Big Gems, Little Gems, ...) gets a percentage of total portfolio budget.
Per-stock cap	A single stock may not exceed a percentage of total budget, by market-cap class: Large 10%, Mid 6%, Small 4%, Micro 2%.

The per-stock cap is **per individual stock**, not a shared pool across all stocks of that cap class. This distinction was originally implemented wrongly — as a shared bucket — which silently skipped legitimate trades once a few small-caps had been bought. Market-cap class comes from AMFI's official half-yearly categorisation, held in `STOCK_CAP_CLASSIFICATION`.

Budget is recycled when a position closes, so the ledger reflects capital actually at risk rather than cumulative spend.

Within those caps, conviction sets the ticket size (2026-08-26 onward):

Lite score	Position
> 92	₹25,000
69 – 92	₹10,000
< 69	not bought

The thresholds are **percentile-matched to the engine that produces them**, and that is the load-bearing detail. A cutoff like "75" is a statement about a score distribution, not about a company. Move it to a differently-shaped distribution and the number survives while the policy silently changes.

This has now bitten three times, in both directions:

Date	Change	If cutoffs had been carried over unchanged
2026-08-26	full engine → lite	₹10,000 band would fall from 36.6% of names to 13.9%
2026-08-26	sizing re-enabled	— recalibrated 85/75 → 85/63
2026-08-29	upside → reachability	₹25,000 band would jump from 7% to 21%

So the cutoffs are re-derived from the share of names each band was meant to capture. At 92/69:

Band	Intended	Achieved
₹25,000	7.2%	7.0%
₹10,000	36.6%	37.2%
not bought	56.2%	55.8%

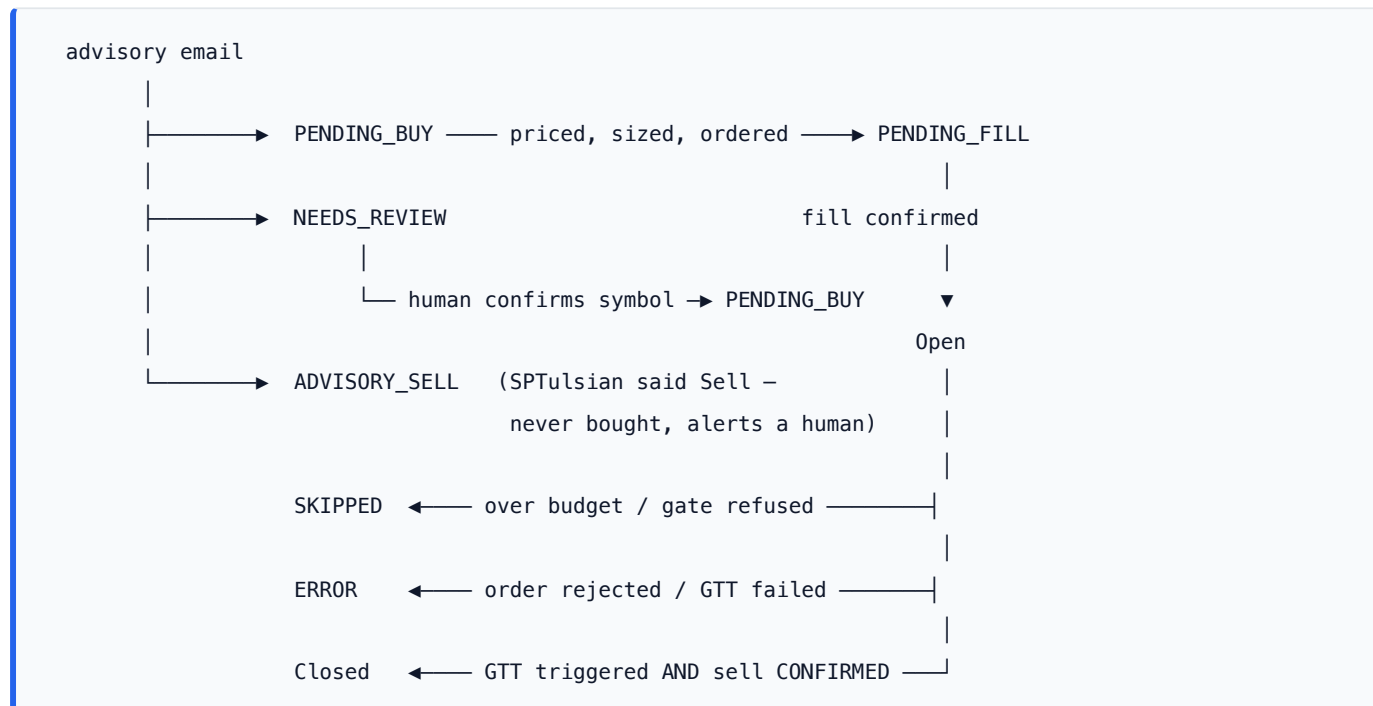
Re-run `tools/recalibrate_bands.py` after any change to the engine's components or weights. Treat these numbers as derived, never as constants — `tests/test_conviction_sizing.py` asserts the *structure* (two descending bands, floor equal to the lower one) rather than the literals, so a legitimate recalibration does not read as a test failure.

Two consequences worth holding onto. First, the upper cutoff rests on about three names out of 43 and is poorly determined; the lower one sits near the median where percentile estimates are stable, so 69 is far more trustworthy than 92. Second, this deploys **more** capital than flat sizing — roughly ₹547k per 100 recommendations against ₹500k — so it is not a risk reduction.

`CONVICTION_MIN_SCORE` is pinned to the lower band and kept at or above `conviction_lite.ACCEPT_FLOOR`, so sizing can never fund a name the engine's own verdict rejects.

None of this is evidence that the score predicts returns. It fixes a distribution mismatch in a policy resting on an unvalidated signal; the argument in §3.5 still stands.

2.3 Trade lifecycle



Closed requires a **confirmed fill**, not merely a triggered GTT. A GTT can leave the active state three ways: it triggered and the sell filled (a real close), it triggered but the day-validity sell order expired unfilled, or it was cancelled. Only the first closes the trade; the other two get a fresh GTT at the same target, tagged with a retry counter, so a position is never left silently unprotected.

PENDING_FILL applies the same rule to the buy side: an order that has been placed but not confirmed filled is not an **Open** position. Reconciliation the next morning either promotes it with the **actual** filled quantity and average price, or requeues it.

ADVISORY_SELL is the one status nothing downstream acts on, which is precisely why it exists. See §4.5.

NON_BUYING_STATUSES in `lib/budget_manager.py` lists every status meaning "no stock was bought". `insert_trade_to_oracle` whitelists against it and falls back to `'Open'`, so a status omitted from that tuple is recorded as a live position holding real shares. Add to it before adding a status anywhere else.

2.4 The dashboard

A Streamlit application at <https://raramdas-stockbot.duckdns.org>, usable from a phone via Add to Home Screen. Login is a password with an encrypted 30-day "remember me" cookie.

Page	Purpose
Overview	Budget deployed, live holdings, current value
Category Drill-Down	Holdings and P&L within one advisory section
Performance	Realised/unrealised P&L, cumulative chart, win rate
Set Targets	Edit target price / timeframe per open trade
Trades Explorer	Filterable full trade history, CSV export
Recommendations	Every tip ever seen, bought or not
Conviction	Evidence-based score per position, with full working
Needs Review	Confirm which instrument an ambiguous tip is. Does not buy
Open Orders	Live Kite orders and GTTs
Classification	AMFI market-cap lookup
Settings & Edits	Budget, allocations, manual trade close

The dashboard cannot place an order. It reads Kite and writes Oracle; no function in `dashboard/` trades.

It used to. Needs Review placed a real buy the moment a human confirmed a symbol, sized from a duplicated `INVEST_AMT = 5000` carrying the comment *"keep in sync if that ever changes"*. It did not stay in sync — sizing moved to conviction bands and that copy kept buying flat — and it bypassed the Have Interest gate and conviction entirely.

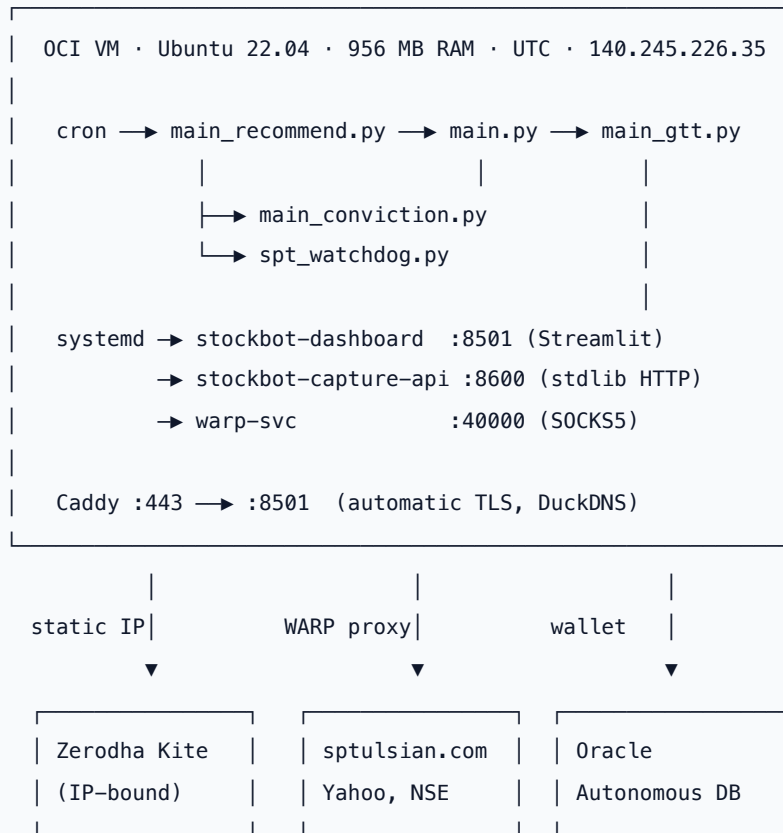
The error underneath was conflating two questions. *"Which instrument is this?"* and *"should we take a position?"* are different, and only the first belongs to a human disambiguating a fuzzy ticker. Confirming a symbol now validates it against a live quote and returns the trade to `PENDING_BUY`, where the normal 11:00 run applies every gate.

That change exposed a trap: `get_pending_buy_trades` windows on `buy_date`, so a tip resolved more than `retry_days` later would rejoin the queue *already outside* the window and never be bought — while still displaying as queued. Hence `resolved_at ( migrations/003 )`, checked alongside `buy_date`. Stamping `buy_date` to today was rejected: it is the date the advisory made the call, and the point-in-time backtest slices price history on it.

Conviction badges are coloured by what a score does, not by a fixed scale. The colour is derived from `lib/bands.py` and gated on the `model` column, so a full-engine score is never painted with lite thresholds. Those thresholds are percentile statements about one distribution; applying them to another is meaningless — banding the full engine's 50–87 range at 92/69 put 69% of the ledger in the "₹10,000" colour for a rule that never applied to it.

3. Technical view

3.1 Topology



Two deployment directories are kept in sync from one Git repository: `/home/ubuntu/stock_bot_v4` (cron jobs) and `/home/ubuntu/stockbot` (dashboard).

3.2 Egress split — the load-bearing constraint

`sptulsian.com` sits behind CloudFront, whose WAF rejects datacenter IP ranges **as a class**. The VM received HTTP 403 with CloudFront error headers on every path, including unauthenticated ones — the origin never saw the request. This is not rate limiting and not bot detection: no user-agent, header set, cookie jar or request pacing changes the outcome, because the discriminator is the source address.

Cloudflare WARP, run in **proxy mode**, egresses from consumer-registered addresses that the WAF serves normally. `warp-svc` exposes SOCKS5 on `127.0.0.1:40000`.

The critical property is **scope**. Zerodha's API is bound to this VM's registered static IP; a system-wide VPN or firewall redirect would move order placement onto Cloudflare's egress and break that binding. So the proxy is attached to exactly one object — the scraper's `requests.Session` — and nothing else:

```
# spt_scraper.build_session()
proxy = os.environ.get('SPTULSIAN_PROXY', '') # socks5h://127.0.0.1:40000
if proxy:
    session.proxies = {'http': proxy, 'https': proxy}
```

Two details carry real weight:

- The variable is named `SPTULSIAN_PROXY`, **not** `HTTP_PROXY` / `HTTPS_PROXY`. `requests` honours the standard names automatically for every module in the process, which would silently route broker traffic through the proxy.
- The scheme is `socks5h`, not `socks5`. The trailing `h` resolves DNS at the proxy, so hostname lookups also traverse Cloudflare rather than leaking from the blocked address.

The one check that matters. If line 1 ever returns a Cloudflare address, the proxy has escaped its scope and broker traffic is at risk:

```
curl -s ifconfig.me # → 140.245.226.35
curl -s --socks5-hostname 127.0.0.1:40000 ifconfig.me # → a Cloudflare address
```

Traffic	Egress	Why
Zerodha orders, holdings, GTTs	Static IP	Broker IP binding — must never move
Gmail IMAP, Oracle	Static IP	No reason to proxy
sptulsian.com, Yahoo, NSE	WARP SOCKS5	The only traffic that is blocked

3.3 Advisory portal scraping

The portal serves its sections through **two different backend controllers**, which requires two parsing strategies:

- **JSON** — the page wires up a per-section endpoint (`/getMWActiveData` for Little Gems, `/getFCActiveData` for Big Gems). The endpoint name is read off each page rather than hardcoded: calling one section's endpoint for another returns HTTP 500, which is easily misdiagnosed as a portal outage.
- **HTML** — the remaining sections ship no data endpoint and render calls directly into the page markup. Row markup is not positionally stable (the same table emits 11- and 13-cell rows, carrying both desktop and mobile cells), so fields are read from row text by content rather than column index.

Three correctness rules govern what is done with a scraped call:

Login is verified on substance, not markers. Checking for a "logout" link passes while signed out — the public teaser page contains that string in its navigation. A session is accepted only if it yields parsed call rows, which no signed-out response can produce.

Archived is not the same as closed. Little Gems and Big Gems carry no active list on this subscription, so even same-day calls appear under "Archives". Closure is determined by an actual exit remark (`Target met...`), not by which table a row sits in. Treating archived as closed would freeze out 82 of 86 open positions.

Calls are matched within their own section. The same stock is called in several sections with different targets and horizons — a Multibagger call may target 2027 at a far higher price than a 3-month Big Gems call on the same stock. Applying the wrong section's target arms a GTT that never triggers and parks the capital.

Regular Income Bluechips is excluded entirely: it is a covered-call section whose "Target" is a total position value (e.g. ₹7,80,000), not a per-share price.

3.4 Conviction engine

Answers a question the advisory does not: *is this call well-supported by public evidence?* Four weighted layers over a 100-point composite.

Layer	Points	Checks
Fundamentals	40	Piotroski F-Score, Altman Z''-EM, Beneish M-Score, ROE, debt/equity, FCF yield, revenue growth
Consensus	25	Analyst rating, target upside, coverage breadth, advisory target sanity
Technical	20	50/200-DMA alignment, RSI overbought guard, 52-week exhaustion, volume confirmation, liquidity
Governance	15	NSE ASM/GSM surveillance, promoter holding, institutional holding, results event risk

Sources: Yahoo Finance (fundamentals, consensus, price history), NSE (surveillance). Both are fetched through the WARP proxy and both fail soft.

Three departures from the conventional design, each driven by what this portfolio actually contains:

Missing data renormalises; it does not derate. The portfolio is small- and micro-cap heavy, and 13 of 37 held symbols have zero analyst coverage. Deducting points for absent coverage would systematically penalise the entire Little Gems category — encoding "*we know less*" as "*this is worse*". Instead the score reports **quality** on what was measurable, and a separate **evidence** figure reports how much of the 100-point frame could be assessed. Layers enter the composite weighted by coverage, so a layer resting on one surviving check carries proportionally less weight rather than extrapolating that check across its whole budget.

UNKNOWN (tried and failed) reduces evidence. **N/A** (the metric never applied) does not.

Financials skip the accrual models. Piotroski, Altman and Beneish assume an operating balance sheet; current ratio, gross margin and asset turnover do not transfer to a lender. Financial Services names mark those checks N/A and renormalise rather than producing confident nonsense.

Gates are separated from warnings. A gate forces recommend-reject; a warning is surfaced and leaves the verdict to the score. Tuned against the live portfolio, where over-eager gates produced obvious false positives:

- **Beneish is a warning, never a gate.** Its sales-growth term over-flags fast growers; on this portfolio it flagged BEL, CG Power, Solar Industries and Mazagon Dock. A gate that rejects those trains the reader to ignore the tool.
- **Altman gates only below 3.0**, not at the 4.15 grey line. Vodafone Idea reads 0.57 and is unambiguous; capital-intensive manufacturers sit under 4.15 structurally.
- **Liquidity and GSM surveillance remain gates.** ₹0.00 crore/day of traded value means a GTT may never fill, regardless of how good the company is.

Below the evidence floor (40 of 100 points) no composite is published at all — a delisted symbol scoring "100/100" off one surviving governance check is worse than reporting nothing.

The FULL engine is display-only — `--engine full` is a research tool and nothing sizes on it. The lite engine (§3.5) is what sizes positions.

That distinction was earned. Sizing was first wired to the *full* engine on 2026-08-25 (>85 → ₹25,000, 75–85 → ₹10,000, below 75 not bought) and reverted the next day: `backtest_conviction.py`, rebuilding scores point-in-time and measuring excess return against NIFTY, found **no detectable relationship** — symbol-level Spearman -0.127 across 37 symbols ($t = -0.76$). The band that would have taken most of the capital performed worst. Two months and 12 realised closes prove nothing either way, which is the point: there was no evidence for a rule that was spending money.

Validation: backfilled across all 86 open positions, it independently flagged both trades already known to be mistakes — `ICDSLTD` (illiquid, ₹0.00 crore/day) and `NIFTY INFRA` (an index, insufficient evidence) — without being told about either.

3.5 The lite engine — what new recommendations actually get

`lib/conviction_lite.py` is the default (`main_conviction.py --engine lite`). Four components, one network call per symbol, no missing-filing holes:

Component	Points	Why it is here
Reachability	40	How likely the target is to be <i>touched</i> inside the horizon. This is the question the exit mechanism actually asks
Momentum 12-1	25	Most robustly documented equity factor globally and in India; the direction the per-check attribution pointed at
Trend alignment	20	The one technical check that correlated positively ($\rho +0.23$)
Liquidity	15	Not alpha — the constraint that decides whether a position can be exited at all

Its shape came from decomposing the full engine. Correlating each individual check against realised excess return at symbol level exposed a contradiction inside the technical layer:

Check	ρ vs excess return
Overbought guard (RSI)	-0.348
Trend alignment	$+0.228$
52-week exhaustion guard	-0.220
Volume confirmation	$+0.138$
Piotroski F-Score	-0.072
Liquidity	$+0.029$
Altman Z''-EM	-0.017

Trend alignment rewarded strength; the two guards penalised the same underlying property. They partly cancelled, which is a large part of why the composite landed near zero.

So the guards became flags, not points. "Don't chase an extended move" is a risk observation about *entry timing*. Encoding it as negative score marked down exactly the names that outperformed in that window. RSI, proximity to the 52-week high, realised volatility and "price already above target" are all surfaced on the dashboard and move no number. Liquidity is the sole hard gate, because an exit that cannot happen is a different risk in kind.

Missing data still renormalises. A recommendation with no scraped target yet drops the reachability component and scores out of the remaining 60, reported as `evidence_pct = 60`. A missing target must not read as "unreachable".

Scores from the two engines are stored with a `model` column and must never be pooled in a backtest. The dashboard badge reads that column too — a score from one engine is never coloured by the other's thresholds (§2.4).

Reachability: why distance alone was measuring nothing

The lite engine originally spent 25 points on "upside to target", scaled across a 0–30% range. Then the targets were measured:

Gap to target, 16 closed trades	
Range	5.82% – 7.69%
Standard deviation	0.58pp
Median, closed vs still-open	6.00% vs 6.05%

SPTulsian sets targets at a near-constant ~6% above their recommended price. A quarter of a composite that was sizing real money was therefore assigning almost the same value to every stock. A term that does not vary across candidates cannot rank them — the same defect as scoring NIFTY's trend, which is identical for everything scored that morning.

What separates a name that reaches target in 3 days from one that takes 23 is not distance but speed. Since the exit is a GTT firing on touch, this is a first-passage problem:

$$z = \text{gap_to_target} / (\text{daily_vol} \times \sqrt{63}) \quad 63 \approx 3 \text{ months of trading}$$

Measured against realised time-to-target on those 16 trades:

Predictor	Spearman	t
Raw gap %	+0.364	+1.46
gap / (vol × √63)	+0.572	+2.61

Because the gap is nearly constant, that z is essentially `1/volatility`: ZEEL at 3.5% daily vol hit target in 3 days; CDSL at 1.76% took 5, 8, 14 and 23. Scale endpoints (full points at $z \leq 0.25$, zero at $z \geq 1.20$) are the p10 and p90 of z across the 33 symbols recommended since the account cutover.

The volatility flag was reworted, not deleted. It previously read as a risk warning — which now contradicts the score on the same page, since the engine rewards the property the flag was penalising. Same inversion as the RSI guard, caught before shipping rather than after.

Two caveats belong next to that +0.572, not in a commit message. Those 16 trades are all winners — the ones that reached target — so the relationship is conditioned on success and is structurally blind to what volatility costs on trades that fail. And with no stop-loss, a volatile name that moves the wrong way is not stopped out, merely held. Rewarding reachability raises hit rate and thickens the tail simultaneously. That trade-off was made deliberately.

reach_z is stored per score (migrations/004) so the model can be measured against realised time-to-target rather than re-derived later.

This is still a hypothesis. Sixteen closed trades, all winners, in a book 26 days old against a 90-day thesis. Nothing has yet had time to fail.

3.6 Component reference

Module	Role
<code>main_recommend.py</code>	Phase 1 — resolve symbols, scrape targets, no orders
<code>main.py</code>	Phase 2 — price, size, buy
<code>main_gtt_oracle.py</code>	Phase 3 — place GTTs, confirm fills, close trades
<code>main_conviction.py</code>	Score today's recommendations; the lite score sets position size
<code>spt_watchdog.py</code>	Liveness alarm for the scraper
<code>spt_selfheal.py</code>	Repairs the scrape path and re-runs the morning, unattended (§4.6)
<code>spt_capture.py</code>	Manual review-then-save of scraped targets
<code>lib/config.py</code>	Env/credentials, <code>KITE_ACCOUNT</code> switch (NEW/OLD), logging, IST
<code>lib/email_reader.py</code>	Gmail IMAP; parses the advisory email into tips
<code>lib/kite_client.py</code>	Kite login (TOTP), symbol resolution, order and GTT placement
<code>lib/order_status.py</code>	Order status lookup and sell-order reconciliation
<code>lib/budget_manager.py</code>	Budget policy, all <code>TRADES</code> reads/writes
<code>lib/spt_scraper.py</code>	Portal login, both parsing strategies, liveness watermark
<code>lib/conviction.py</code>	Four-layer fundamentals engine (<code>--engine full</code>)
<code>lib/conviction_lite.py</code>	Reachability/momentum/trend/liquidity engine (default)
<code>lib/bands.py</code>	Sizing thresholds and <code>size_for()</code> — the single definition both the buy path and the dashboard read
<code>lib/sheet_logger.py</code>	Google Sheets mirror (legacy, still written)
<code>dashboard/app.py</code>	Streamlit UI, all pages
<code>dashboard/db.py</code>	Dashboard's Oracle data access
<code>dashboard/kite_data.py</code>	Multi-account Kite sync, symbol resolution for Needs Review. Places no orders
<code>dashboard/capture_api.py</code>	Authenticated endpoint for <code>spt_capture.py</code>
<code>dashboard/theme.py</code>	CSS design system, table and KPI rendering
<code>tools/recalibrate_bands.py</code>	Re-derives sizing cutoffs from the current score distribution
<code>tools/dryrun_sizing.py</code>	Previews what the buy run would do, with every write tripwired
<code>tools/md2pdf.py</code>	Regenerates the committed <code>README.pdf</code> / <code>ARCHITECTURE.pdf</code>
<code>provisioning/warp_keeper.sh</code>	Recycles <code>warp-svc</code> before it degrades (§4.4)
<code>provisioning/sample_warp_memory.sh</code>	Records <code>warp-svc</code> memory every 30 min — the leak-rate evidence
<code>provisioning/install_crontab.sh</code>	Installs <code>provisioning/crontab</code> ; <code>--check</code> reports drift
<code>backtest_conviction.py</code>	Point-in-time scoring vs realised excess return
<code>archive/</code>	Superseded and one-off scripts; nothing scheduled

Module	Role
bot/	Multi-tenant variant; keeps its own module copies, not scheduled

Entrypoints sit at the repository root because cron invokes them by bare filename; everything they share lives in `lib/`. `bot/` deliberately does not import from `lib/` — it is a self-contained multi-tenant tree.

3.7 Data model

Oracle Autonomous Database, wallet authentication.

Table	Contents
TRADES	The ledger. One row per recommendation, whatever its outcome.
PORTFOLIO_BUDGET	Total capital under management
CATEGORY_ALLOCATION	Per-section allocation % and per-cap-class caps
STOCK_CAP_CLASSIFICATION	AMFI market-cap categorisation
CONVICTION_SCORES	Score history; full working stored as JSON
KITE_HOLDINGS_SNAPSHOT	Holdings per account (<code>account_label</code> NEW/OLD)
KITE_GTT_SNAPSHOT	Live GTTs per account
KITE_ORDERS_SNAPSHOT	Recent orders per account
KITE_SYNC_LOG	Per-account last-synced timestamps

`TRADES` is the single source of truth. Every recommendation writes a row regardless of outcome, which is what makes the Recommendations page a complete record rather than a survivor-biased one.

Schema changes live in `migrations/`, numbered and idempotent (each re-runs safely, tolerating `ORA-01430 / ORA-00955`):

Migration	Adds	Why
002	<code>conviction_scores.model</code>	Lite and full scores are not comparable and must never be pooled in a backtest
003	<code>trades.resolved_at</code>	A tip resolved days after its call must rejoin the buy queue; <code>buy_date</code> cannot be moved without falsifying the record
004	<code>conviction_scores.reach_z</code>	The quantity to validate against realised time-to-target. Kept out of <code>layers_json</code> so it can be correlated directly
005	<code>trades.spt_*</code>	Advisory context at the call — captured because the portal shows only what is live

The `spt_*` columns record what SPTulsian said, and nothing scores on them yet:

Column	Content
spt_market_price_at_call	Market price when the call was made, as distinct from recommended_price — the gap between them is their own margin of safety, and it varies (BHEL called at 434 into a 430.5 market, TD Power at 741 into 752.7)
spt_below_reco	A flag SPTulsian computes themselves
spt_direction	Buy / Sell . Safety-critical — see §4.5
spt_rationale	Their written reasoning. Plain text only for Medium Term ; Little Gems and Big Gems ship it as a base64 PNG (110–160 KB), so it stays null for the sections covering nearly every trade. No OCR is attempted

3.8 Schedule

The canonical schedule is **provisioning/crontab** , installed with `bash provisioning/install_crontab.sh` . Edit that file, not `crontab -e` .

It lived only on the VM until 2026-09-02, which meant a rebuild would have lost it silently and a hand-edit could drift from this document with nothing to compare against. `install_crontab.sh --check` answers "is the live schedule still what the repo says?" without touching anything, and exits non-zero if not — it compares the schedule lines only, so a comment edit is not drift.

The VM clock is **UTC**; IST is UTC+5:30.

Ten entries: seven trading jobs, three maintenance loops.

Job	Cron (UTC)	IST	Purpose
main_recommend.py	0 4 * * 1–5	09:30	Phase 1 — symbols, targets, backfill (§4.6). No orders
spt_selfheal.py	30 4 * * 1–5	10:00	If the 09:30 scrape failed: repair and re-run, before the buy
main_conviction.py	45 4 * * 1–5	10:15	Scoring, lite engine. A failure holds buying
spt_watchdog.py	15 5 * * 1–5	10:45	Pass 1 — are the buy inputs ready?
main.py	30 5 * * 1–5	11:00	Phase 2 — price, size, buy
spt_watchdog.py	45 5 * * 1–5	11:15	Pass 2 — did the buy actually happen ?
main_gtt_oracle.py	30 10 * * 1–5	16:00	Phase 3 — GTTs, confirm fills, close
warp_keeper.sh	*/20 * * * *	—	Recycle the tunnel before it degrades (§4.4)
sample_warp_memory.sh	*/30 * * * *	—	Leak-rate evidence
DuckDNS refresh	*/5 * * * *	—	Dynamic DNS

The ordering is the point. Every repair runs **before** the thing it protects: self-heal at 10:00 precedes scoring at 10:15, which precedes buying at 11:00. A repair that ran after the buy window would be a report, not a recovery.

The watchdog runs twice for the same reason. Everything before 11:00 verifies the buy's *inputs*; none of it can see a run that had good inputs and then died mid-flight, which still logs "Buy Phase complete". That is how two days of buying were lost on 2026-09-01/02.

4. Failure handling

4.1 The silence problem

A dead scraper and a quiet trading day look identical from outside. The buy flow deliberately continues when a scrape fails — recording the recommendation and resolving its symbol matter more than the target, which can be filled in later — so nothing in the pipeline raises. Without a dedicated check, the system could run blind for days while every log line still read "complete".

The fix is a **liveness watermark**: a timestamp written only after a section was genuinely fetched *and* parsed, never on a mere attempt. It is a signal a failed fetch cannot fake. Judging freshness from the newest row in `TRADES` was rejected — a genuinely quiet week would then raise a false alarm.

`spt_watchdog.py` applies two independent rules and emails on either:

Rule	Condition	Rationale
Absolute	Watermark older than 30 h	Backstop for outages spanning a day
Trading-day	Past 10:30 IST on a weekday, watermark must carry today's date	The absolute rule alone would not fire until the next evening — after a whole session had run blind

The scrape runs *before* the no-tips early return, making the watermark a true daily heartbeat. Otherwise a market holiday — a weekday with no tips — would be indistinguishable from an outage and would cry wolf every time.

4.2 Diagnostic matrix

Symptom	Likely cause	Check
Connection refused to <code>127.0.0.1:40000</code>	WARP down	<code>sudo systemctl status warp-svc; warp-cli status</code>
HTTP 403, CloudFront headers	Egress blocked or proxy bypassed	Compare direct vs proxied <code>ifconfig.me</code>
HTTP 500 from a section endpoint	Wrong controller for that section	Endpoint is read per page — confirm the page still declares it
Pages fetch, zero rows	Portal markup changed	<code>python3 spt_capture.py --dry-run</code>
A trade shows <code>Open</code> with shares you do not hold	Holdings attributed to the wrong lot	Compare ledger <code>SUM(my_buy_qty)</code> per symbol against Kite; see §4.5
A resolved Needs Review tip never gets bought	Outside the <code>buy_date</code> window	Check <code>resolved_at</code> is populated (migration 003)
Score changed sharply with no market move	Engine components changed without recalibration	<code>python3 tools/recalibrate_bands.py</code>
Login "succeeds", no data	Credentials changed	Substance check should catch it; verify <code>SPT_USERNAME / SPT_PASSWORD</code>
TCP connects but nothing responds on :22 and :443	Global OOM — userspace wedged, kernel still answering	OCI console → force reboot, then <code>journalctl -b -1 \ grep -i oom</code>
Scrape fails, yet <code>warp-cli status</code> reads "Connected / Network healthy"	Tunnel degraded , not down — the daemon's self-report is wrong	<code>bash provisioning/warp_keeper.sh --check</code> , which fetches rather than asks. See §4.4
<code>warp-svc</code> restarting repeatedly	Memory leak hitting its cgroup cap	<code>systemctl show warp-svc -p MemoryCurrent</code> ; this is the cap working
Trade stuck <code>PENDING_BUY</code> for days with a blank <code>have_interest</code>	Its scrape failed on the day; retries re-attempt the buy, not the lookup	Fixed by <code>backfill_missing_interest()</code> — see §4.6

4.3 Safety invariants

These hold by construction and should be preserved by any future change:

1. Broker traffic egresses from the registered static IP. Always.
2. A symbol that does not resolve cleanly is never bought — it becomes `NEEDS_REVIEW` and waits for a human.
3. A trade is `Closed` only on a **confirmed** sell fill, never on a GTT status alone.
4. Conviction sizing is on, and its thresholds are **percentile-matched to the engine that produces them**. Cutoffs are statements about a score distribution, not about the world; moving them between engines unchanged silently changes the policy. See §2.2 and `lib/bands.py`.
5. **The dashboard never places an order.** Every buy goes through `main.py`, so there is exactly one code path that spends money and exactly one place the gates can be applied.

- 6. **A quantity is never claimed from a symbol-level lookup.** Holdings and order lists are per symbol; positions are per lot. Anything inferring "this order filled" from a holding must first subtract what other open lots already claim. See §4.5.
- 7. **An explicit SELL is never bought**, from either the email or the portal, and always reaches a human.
- 8. Thresholds, budgets and status lists are defined **once**. `lib/bands.py` holds the sizing cutoffs; `NON_BUYING_STATUSES` holds the statuses that mean no stock was bought. Every duplicated constant in this system has eventually drifted.

4.4 Resource containment

The VM has **956 MB of RAM**. That is the binding constraint on everything here, and it is why the box went down on 2026-08-26.

`warp-svc` leaks. The kernel OOM-killed it on 18, 20, 22 and 24 August — roughly every two days, at ~250 MB resident each time — and on the 26th it grew fast enough to wedge the machine before the OOM killer resolved it.

The failure mode is worth understanding, because the symptom is misleading. With no cgroup limit, `warp-svc` 's growth triggers a **global** OOM, and the kernel then picks victims anywhere in the system. It took out `sshd` and Caddy. From outside, both ports still completed a TCP handshake — the kernel was healthy — but no daemon ever replied. Two unrelated services accepting connections and answering nothing is the signature of userspace starvation, not of a network or application fault. The disk, the obvious first suspect, was at 22%.

Three changes contain it:

Change	Why
<code>MemoryMax=300M</code> , <code>MemoryHigh=220M</code> , <code>Restart=always</code> on <code>warp-svc</code>	Converts a global OOM into a local one. <code>systemd</code> kills only that unit and restarts it in 5s; the kernel never gets to choose <code>sshd</code> . Steady state is ~80 MB.
2 GB swapfile, <code>vm.swappiness=10</code>	There was none. Swap is an emergency cushion here, not a working tier — hence the low swappiness.
<code>SystemMaxUse=200M</code> on <code>journald</code>	<code>warp-svc</code> dumps full metrics structures line by line when it degrades; journals had reached 632 MB, uncapped.

The proxy is needed for about a minute a day, so a restart is invisible to the workload. If `warp-svc` ever begins restarting frequently, that is the cap doing its job — the leak is upstream in Cloudflare's daemon, not in this system.

Containment was necessary but not sufficient, because the daemon degrades before it dies. The cap only acts at 300 MB. On 2026-09-18 `warp-svc` sat at 230 MB with 1,683 tunnel-disconnect lines in a single day, below the cap and therefore never restarted — while `warp-cli status` reported "Connected / Network healthy" and the SOCKS proxy returned **HTTP 000** to every request. `warp-cli connect` did not help. `systemctl restart warp-svc` did: memory fell to 75 MB and the proxy answered 200 in three seconds.

So there is a window, roughly 180–300 MB, in which the tunnel is useless and every health signal short of a real request says otherwise:

Signal	What it actually reports
<code>systemctl is-active</code>	The process exists
<code>warp-cli status</code>	What the tunnel <i>believes</i> about itself
A real fetch through the proxy	Whether the scraper can work

Only the third is load-bearing, and `warp_keeper.sh` is built on it — it restarts when a genuine fetch of the portal fails, or pre-emptively once memory reaches **180 MB**, which is inside the degradation window but below the cap. Recycling early is cheaper than detecting each failure afterwards.

Measured growth is **~32 MB/day**: 61 MB after a restart on 2026-09-18, then 73 MB at ten hours and 110 MB at thirty-seven. So the keeper recycles roughly every four days and the daemon never reaches the state that caused the outage.

Two earlier figures in this document were wrong, in opposite directions, and both for the same reason — a window too short or too dirty to extrapolate from. The first compared against a four-day-old restart and was far too high; the second read ~28 MB/day off a single ten-hour window. Quote this number with the window it came from, and re-derive it from `provisioning/sample_warp_memory.sh` rather than from two points.

This reframes the incident history. 2026-08-26, 09-04, 09-17 and 09-18 all correlate with high `warp-svc` memory. The reconnect-on-failure logic added on 09-08 was treating a symptom: it succeeded at 09:30 on 09-18 and the tunnel was degraded again by that evening. Reconnecting a degraded daemon does not repair it — only replacing the process does.

4.5 Two failures worth naming

Attributing a symbol's holding to one order. When an order appears in neither today's list nor order history, `get_order_status` can infer a previous-day fill from holdings. It used to return the *entire* holding as that order's filled quantity, which is only correct when the symbol is held in one lot.

On 2026-08-28 that marked a 3-share BSE order COMPLETE for 8 shares. All eight belonged to eight earlier one-share trades; Kite held exactly 8; the ledger then claimed 16. The order had not filled at all, and ₹26,608 of stock that was never bought sat in the ledger inflating deployed capital.

The fallback now requires the caller to supply how many shares are **not** already claimed by other open lots, caps the claim at what the order requested, and returns `NOT_FILLED` when the holding is fully explained — a positive statement rather than an absence of information. Without that context it declines to infer.

This is the second instance of one pattern: the IDEA sell-order mis-attribution matched on symbol + quantity alone and closed two trades against a single fill. Hence invariant 6.

A SELL call that nobody sees. The email pattern hardcoded `(Buy @ price)`, so `Call added: X (Sell @ N)` did not match. The bot would not have bought it — but the tip was dropped at the regex, never logged, never recorded. The failure mode was silence: a position held with a GTT resting at a target the advisory had just withdrawn, and nothing anywhere saying so.

Direction is now read from the email **and** from every portal section, and either saying Sell is enough. The JSON sections expose a `buy_sell` field; the HTML sections render the call as `<Direction> @ <price>` in their mobile cell, which one pattern reads alongside the price and timestamp. Coverage is 100% of live and archived rows across all five sections.

On archived rows the exit remark is skipped when locating the call: a remark reading "Exited: Sell @ 1,600" would otherwise be parsed as a sell call at the exit price, inventing both a direction and an entry price from the way the position was closed. The span is skipped rather than the text truncated, because truncating lost the direction on every archived row whose remark happened to precede the call text — 23 of 43 in Medium Term. The call becomes `ADVISORY_SELL`: never bought, inert to every buy query, and surfaced three ways — the recommend log, an amber pill on the dashboard, and the watchdog's alert mail, which reports how many shares are still held. "They said exit" and "they said exit and you own 340 shares" need different responses.

A **blank** direction deliberately does not block: most HTML sections carry no direction field, so treating blank as a sell would stop nearly every buy. Only an explicit Sell blocks.

Nothing in the pipeline can close a position, so a signal only a human can act on has to reach a human — that is the whole purpose of this status.

4.6 A trade that could never recover

A tip whose 09:30 scrape failed was recorded `PENDING_BUY` with a blank `have_interest` and held — correctly, since buying without knowing whether the call is still live would be worse. The retry logic then re-attempted the **buy** every morning, and nothing ever re-attempted the **lookup**: this job reads *today's* emails, and a trade held from an earlier day is not in them.

So the missing data could never arrive. Trades #821–824 sat that way from 2026-09-17, retried daily, reported by the watchdog each morning, until the buy window expired and they aged out unbought. The system was neither broken nor recovering; it was busy failing in a loop.

`backfill_missing_interest()` closes the gap. The scrape cache already holds every live call, so matching a held trade against it costs nothing beyond the query. It runs **before** the no-tips early return, because a quiet Monday is still a day Friday's held trades should get their data — placed after it, a day with no new tips would strand them for another day. It filled 7 of 7 on its first live run.

Only blank rows are touched. A recorded "No Interest" is an answer, and must never be overwritten by a later lookup that happens to miss.

5. Operations

5.1 Deploy

```
# local
git add -A && git commit && git push

# VM — both directories track the same repo
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  cd /home/ubuntu/stock_bot_v4 && git pull -q
  cd /home/ubuntu/stockbot && git pull -q
  sudo systemctl restart stockbot-dashboard
  cd /home/ubuntu/stock_bot_v4 && git status --porcelain'
```

That last line is the one that matters. **git pull is the only supported deploy**. Copying a file with `scp` works, and that is the problem: the VM then runs code that exists nowhere else, and `git status` is the only thing that says so.

Between 2026-09-08 and 09-19 the VM ran ten days of `scp`-deployed changes against a repo whose `HEAD` was from 09-02, because a push was blocked on expired credentials and the deploys carried on anyway. Nothing failed. The cost was epistemic: there was no way to answer "is production running the fix?" without checksumming files by hand, and a single `git checkout .` by anyone would have destroyed work that existed on one laptop and one VM.

If the working tree is ever dirty, reconcile it rather than leaving it: back it up, compare every dirty file against the commit, and abort if any differs — a mismatch means something was edited on the VM that is not in the repo, and that is a decision for a human, not a silent overwrite.

5.2 Health check

```
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  curl -s ifconfig.me; echo                # must be 140.245.226.35
  curl -s --socks5-hostname 127.0.0.1:40000 ifconfig.me; echo # must be Cloudflare
  sudo systemctl is-active warp-svc stockbot-dashboard stockbot-capture-api
  free -m                                  # swap must be non-zero
  cd /home/ubuntu/stock_bot_v4
  bash provisioning/warp_keeper.sh --check # fetches; the only real proxy check
  bash provisioning/install_crontab.sh --check # schedule drift
  python3 spt_selfheal.py --check-only      # would it need to repair anything?
  python3 spt_watchdog.py --check-only      # never sends mail
  git status --porcelain'                  # MUST be empty — see §5.1
```

On a 956 MB VM the memory line is not incidental — see §4.4.

`warp_keeper.sh --check` replaces reading `MemoryCurrent` by eye. It reports memory *and* performs a real fetch, and exits non-zero if it would act — which is the distinction §4.4 is about, since the daemon calls itself healthy in precisely the state that breaks the scraper.

5.3 Testing convention

Anything that touches money or the ledger is exercised in an isolated directory with Oracle writes, Sheet writes, and order placement monkey-patched out, run against **real live data**, and inspected before the real run.

Mocked-write test harnesses have caught, among others: the pooled-budget bug, the GTT close that recorded no sell price, and a `DataFrame.__bool__` call that silently nulled every financial statement in the conviction engine.

Automated trading agents do not place real trades on the operator's behalf during development. Commands that move money are handed to the operator to run.

6. Known limitations

- **WARP is a single point of failure** for the advisory feed, and a consumer service with no availability guarantee. Mitigated, not eliminated: `warp_keeper.sh` recycles the daemon before it degrades, `spt_selfheal.py` repairs and re-runs the morning before the buy window closes, the email channel still delivers call headlines, the watchdog surfaces an outage within one trading morning, and `spt_capture.py` remains a manual break-glass path. All of that is redundancy around one consumer tunnel, not a second route to the data.
 - **Yahoo Finance is an unofficial source** for fundamentals. Coverage is good for this portfolio (all 37 held symbols return 4–5 years of statements) but is neither contractual nor audited.
 - **Conviction governance is partial.** Promoter *pledge* and holding *trend* need shareholding-pattern filings, which are not yet parsed; only point-in-time holding percentages are scored.
 - **Analyst consensus is thin for micro caps** by nature. Handled by renormalisation, but it means the Consensus layer contributes little for much of Little Gems.
 - **Single tenant in practice.** `TENANT_CONFIG` and the `bot/` tree carry multi-tenant scaffolding that is not currently scheduled.
 - **There is no stop-loss and no time-based exit.** A position that never reaches its target is simply held. This is deliberate for now, but it means the only way a trade leaves the book is by succeeding — which is why the ledger's win rate is not evidence of anything (§7) and why favouring volatility carries an unmeasured cost.
 - **The advisory's written rationale is unavailable for 96 of 101 positions.** Little Gems and Big Gems serve it as a base64 PNG rather than text. Only the Medium Term section exposes prose. OCR was considered and rejected: two lossy stages before any signal, against a format the publisher appears to have chosen deliberately.
 - **The conviction model cannot yet be validated.** Sixteen closed trades, all winners, in a book four weeks old against a 90-day thesis. `reach_z` is stored so the test becomes possible later; it is not possible now.
-

7. Note on the conviction layer

The conviction engine computes published metrics from public data and shows its working. It is decision support, not financial advice, and not a prediction. Every threshold in it is a convention rather than a fact, tuned against one portfolio and re-derived three times already.

It now sets position size, which raises the stakes on being honest about what it is worth. The record: the full engine's composite showed **no detectable relationship** with excess return across 37 symbols. The lite engine's reachability component correlates with time-to-target at rho +0.57, but on 16 closed trades that are all winners, in a book four weeks old against a 90-day thesis. That is a well-motivated hypothesis, not a validated model.

Two structural cautions that no amount of further fitting will remove:

- **The sample only contains successes.** Positions close when they touch target; ones that fail simply stay open. Any model fitted to closed trades is fitted to the easy half of the distribution.
- **Optimising for speed-to-target favours volatility**, and with no stop-loss the same property makes losing positions worse. Hit rate and tail risk move together here, and only the first is currently measurable.

reach_z is stored per score so the honest test becomes possible once enough of the book resolves. Until then, every score is shown alongside the checks that produced it, and the judgement stays with the human reading the output.